Design a URL Shortener - Step 3: Design Deep Dive

1. Overview

- Deep dive into:
    - Data model
    - Hash function
    - URL shortening
    - URL redirecting

2. Data Model

Problem with Hash Table

- In-memory hash table:
    - Not scalable
    - Memory is expensive
    - Cannot store hundreds of billions of records

Solution: Relational Database

- Store mapping in database:

```
None
<shortURL, longURL>
```

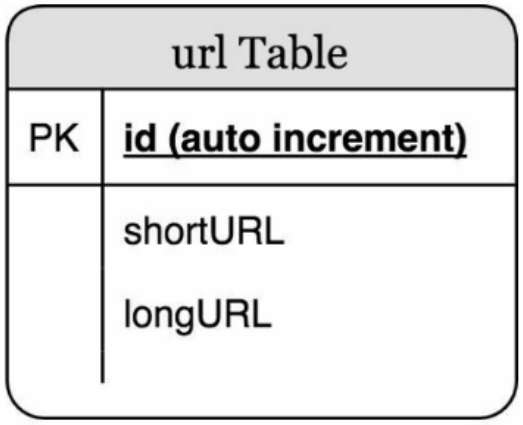

Table Structure

```
None

id | shortURL | longURL
```

Key Points

- `id` → unique identifier
- `shortURL` → indexed for fast lookup
- `longURL` → original URL

3. Hash Function

Purpose

```
None

longURL → hashValue (short URL key)
```

Character Set

```
None
[0-9, a-z, A-Z] → 62 characters
```

Capacity Requirement

- System must support:

```
None
≈ 365 billion URLs
```

Hash Length Calculation

Find smallest **n** such that:

```
None
62^n ≥ 365 billion
```

Result

```
None
n = 7
```

Because:

```
None
62^7 ≈ 3.5 trillion
```

Conclusion

- Hash length = **7 characters**
- Enough to support required scale

4. Hashing Approaches

1. Hash + Collision Resolution

- Apply hash function on long URL
- Generate hashValue
- Handle collisions if occur

2. Base62 Conversion

- Convert numeric ID → base62 string
- Deterministic and collision-free

5. Key Takeaway

Design decisions:

- Use **database instead of memory**
- Use **7-character hashValue**
- Choose between:
    - Hash + collision handling
    - Base62 encoding